

23/July/24:-

Data structures (DSA) :- in C++ & C and Algorithm

(CER3C3)

Date: / / Page:

Activities (Technical)

- ↳ DBMS
- ↳ Operating system
- ↳ DSA
- ↳ Computer system

- ↳ Smart India Hackathon (SIH)
- ↳ Internal hackathon
- ↳ ICPC → ACM (ICPC)
- ↳ International college pageant contest

MST → 1, 2, 3

Tough papers

↳ Quiz → on applications → programs.

{ CODE SHEFF } → Prog. Platforms
and many more

↳ GDSC → Google development student club

(Technical club)

Google classrooms :-

↳ IDE :- VS code, and many more
↳ command line run

↳ Recommended Books of DSA :-

- ① Mark Allen Weiss → "Data structures and Algo. Analysis in C++"
- ②
- ③
- ④
- ⑤
- ⑥
- ⑦

↳ Good Books { for course }

- ① Fundamental of data structure, E. Horowitz & S. Sahani
- ✓ ② Fundamental of data structures in C++, E. Horowitz & S. Sahani
- ③ E. Balaguruswami in C programming
↳ Based on C.

24 July 24:-

Data Structure (DSA):-

① Assignment

HW

Ques →

$A[] = \{2, 4, 6, 8, 10\}$

$B[] = \{1, 3, 5, 7, 9\}$

$C[] = \{1, 2, \dots\}$

Take i, j, k as arrays:
compare i & j → the smallest
value → comes in (c) &
the pointer shifts further
do same → again

Sol:- Logic:-

- ② Assignment { → Later on after → Sorting Methods } } HW
③ Assignment → Binary search
④ Assignment → Matrix multiplication

Sort :-

- ① Insertion sort → Insertion sort is stable, as the values appear on the same places after sorting as they were before sort.

$A = \{14, 12, 1, 6, 17, 5, 19, 3\}$

Step ① 12 14

Step ② 1 12 14

→ Property techniques of any sort

→ without use of any extra memory, you are performing sorting known as in-place sorting.

→ Extra Buffer { Not applicable }
→ Extra memory { Not possible }

→ In your unsorted array you can have duplicate element, The order in which duplicate keys are appearing after sorting too they were visible on the same position only → Stable Sorting

Logic →

```
for (i = 1; i < n; i++) {
    int key = a[i];
    j = i - 1;
```

```
    while (j >= 0 && a[j] > key) {
        a[j + 1] = a[j];
        j--;
    }
}
```

② Selection sort → ^{in-place sort} _{Not-stable algorithm}

A[] = { 14, 12, 1, 6, 17, 5, 19, 3 }

1st pass = 1, 12, 14, 6, _____

1, 3, 14, ..., 12 Table is on

consider 1st element as min. & all the
 → compare 1st element with ~~other~~ values then
 swap the two values, & the sort goes on
 when you find the 1st element is
 lesser than the values (upcoming).

Logic →

```
for (i = 0; i < n - 1; i++) {
    for (j = i + 1; j < n; j++) {
```

25/July/24:-

LAB ASSIGNMENT-1
+hw

3 August
11:59
zip file

Data Structures (DSA):-

File Handling (HW)
in c++

Debugging a program (HW)

③ Algorithm → Bubble sort:-

program file = input.txt
output.txt

→ Taken an unsorted array.

14 12 1 10 6 4 15 3

Iteration 1: 12 10 6 4 14 3 15

→ Compare every value with nearest value → if smaller → swap.

→ It 2: 1 10 6 4 12 3 14 15

→ It 3: 1 6 4 10 3 12 14 15

→ It 4: 1 4 6 3 10 12 14 15

→ It 5: 1 4 3 6 10 12 14 15

→ It 6: 1 3 4 6 10 12 14 15

Logic:-

```
for (i=0; i < n-1; i++) {  
    for (j=0; j < n-i-1; j++)  
    {  
        if (arr[j] > arr[j+1])  
            swap(arr[j], arr[j+1]);  
    }  
}
```

→ optimization → already sorted array — { check }

Logic:-

```
for (i=0; i < n-1; i++) {  
    bool flag = false;  
    for (j=0; j < n-i-1; j++)  
    {  
        if (arr[j] > arr[j+1])  
            swap(arr[j], arr[j+1]);  
        flag = true;  
    }  
    if (flag == false) {  
        break;  
    }  
}
```

→ used → flag for optimization

- Efficiency: - Evaluation of a program
 - Better Algorithm → execution time less → more efficient
 - on paper → How to check

→ Data structures (Learn or Read):-

- ① array
- ② stack
- ③ Queue
- ④ link list
- ⑤ tree & Graph
- ⑥ trees
- ⑦ array table
- ⑧ composite data structure

→ Data structure (Learn) → condition (analyse) → Acc. to that → apply data structure type → to make your program → consume less time → more effective.

- write an Algorithm / ^{code} { Preference, code }
 - write a program
 - write a procedure (English steps)
 - write a pseudocode
- } Based upon Logic
- MST's, end sem
→ can ans → ques → this way.

[→ every statement → Holds How much feed. → tells efficiency of Algorithm.]

Ques → write code to generate fibonacci series / n^{th} term print. (upto n^{th} term print)

Sol:- → iterative version

0 1 1 2 3 5

→ started from zero (0)

→ if $n < 0$: invalid

→ for $n < 0$ → statement → How many time your code run:-



for
 $n = 0, 1 \rightarrow$ statement $\rightarrow$ How many times
 code runs

for
 $n > 1 \rightarrow$ statements $\rightarrow$ How many times
 code runs